DESIGN AND ANALYSIS OF ALGORITHMS · DAA

Dijkstra’s Shortest Path Algorithm

From weighted graphs to a guaranteed shortest route.

Student: Aanya Mishra · prn: 25070521279 · sem/Section: 3/E-2

Generated using Gamma



Made with GAMMA

THE CORE IDEA

Shortest paths turn weighted graphs into decisions

At every stage, the algorithm asks one practical question: which reachable vertex can now be settled safely?

Find the minimum

Finds the minimum-cost path from one source vertex to every reachable vertex in a weighted graph.

Keep the route

Returns both the best-known distance and the predecessor chain used to reconstruct the route.

Reason over networks

It is a core single-source shortest-path idea in graph algorithms and network reasoning.

Made with GAMMA

GRAPH VOCABULARY

A graph is a map of vertices connected by weighted edges

Vertex

An object or location such as A, B, or C.

Edge

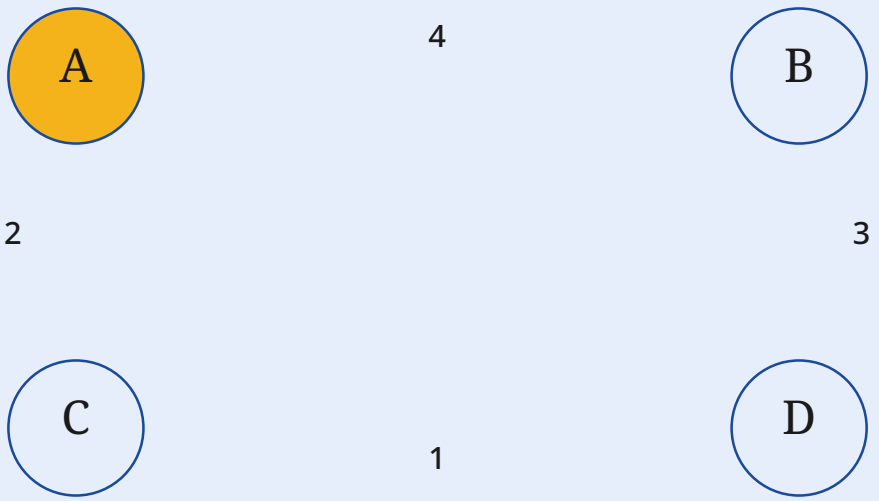
A connection between two vertices.

Weight

The cost of using an edge: distance, time, or delay.

Source and destination

Where the route starts and where it ends.



Weight = edge cost. A weighted graph records both connectivity and cost.

Made with GAMMA

WHY THE GREEDY STEP WORKS

Dijkstra makes a greedy choice that stays safe

Choose the smallest tentative distance

Greedy algorithms choose the best-looking option available at each step. Dijkstra selects the unvisited vertex with the smallest tentative distance.

Non-negative weights make the choice safe

With non-negative edge weights, that distance cannot later be improved by going through a farther vertex. Negative edge weights break this guarantee; Bellman–Ford is the alternative for graphs that may contain them.

Made with GAMMA

STEP 1 · SET THE STATE

Initialization records what is known and what is not

Before the first choice, every vertex receives a safe default.

1. Input: a weighted graph and a source vertex s .
2. Set $distance[s] = 0$; set every other distance to ∞ .
3. Mark every vertex unvisited.
4. Set $predecessor[v] = NIL$ until a best preceding vertex is found.

Remember: The distance array stores current best costs; the predecessor array reconstructs the final path.

Made with GAMMA

STEP 2 · UPDATE THE STATE

Relaxation improves a route one edge at a time

Each pass tests whether reaching a neighbor through the current vertex is cheaper.

01

Extract

Choose the unvisited vertex with the smallest tentative distance.

02

Test

For each unvisited neighbor, compute $distance[u] + weight(u, v)$.

03

Decide

If the candidate is smaller, update $distance[v]$ and $predecessor[v]$.

04

Record

Mark u visited and repeat until the destination is settled.

Decision point

Shorter route found? Yes → replace the distance and predecessor. No → keep the current record.

Made with GAMMA

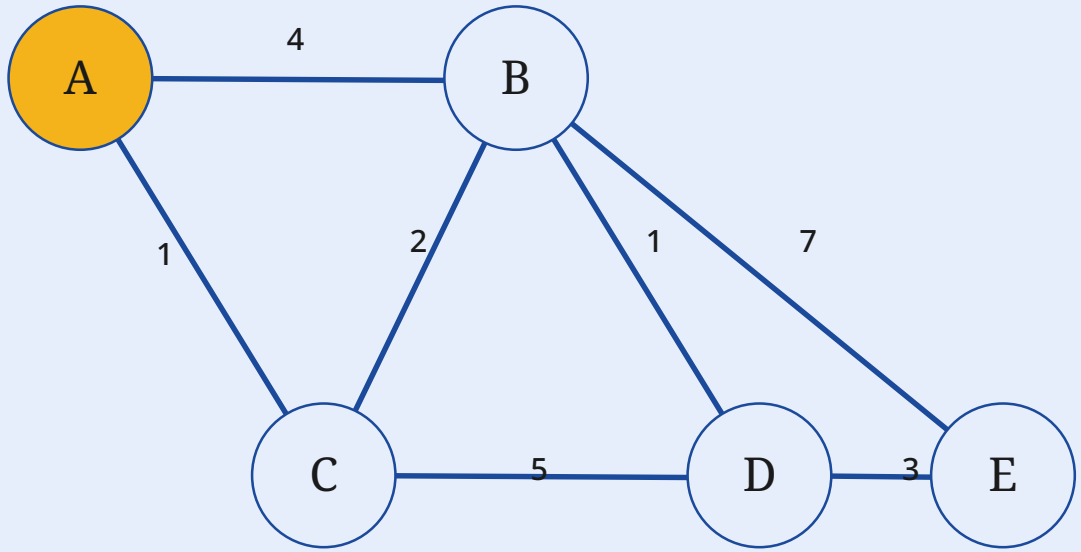
WORKED EXAMPLE

Example: route from A to E

Use an undirected weighted graph with five vertices and these seven edges.

Edges: A–B = 4 · A–C = 1 · C–B = 2 · B–D = 1 · C–D = 5 · D–E = 3 · B–E = 7

Source: A · **Destination:** E



The final route will be highlighted after the distances settle.

Made with GAMMA

THE DISTANCES SETTLE

The shortest route settles at cost 7

Start with A = 0 and all other distances = ∞. Process vertices in this order: A, C, B, D, E.

Processed	A	B	C	D	E	Key update
Start	0	∞	∞	∞	∞	Source A = 0
A	0	4	1	∞	∞	A→B, A→C
C	0	3	1	6	∞	C→B improves 4→3; C→D = 6
B	0	3	1	4	10	B→D improves 6→4; B→E = 10
D	0	3	1	4	7	D→E improves 10→7
E	0	3	1	4	7	Destination settled

FINAL PREDECESSOR CHAIN

E ← D ← B ← C ← A

Shortest path: A → C → B → D → E

Total cost: 1 + 2 + 1 + 3 = 7

Made with GAMMA

Pseudocode : repeat extract, relax, and record

Initialize safe defaults; select the next settled vertex; test each outgoing edge during relaxation; predecessors rebuild the route.

```
DIJKSTRA(G, source):
  for each vertex v in G:
    distance[v] ← ∞; predecessor[v] ← NIL; visited[v] ← false
  distance[source] ← 0
  while there is an unvisited vertex:
    u ← unvisited vertex with minimum distance[u]
    visited[u] ← true
    for each neighbor v of u:
      candidate ← distance[u] + weight(u, v)
      if candidate < distance[v]:
        distance[v] ← candidate
        predecessor[v] ← u
  return distance, predecessor
```


Made with GAMMA

COMPLEXITY AND LIMITS

Efficiency comes from the graph representation

The data structure determines how quickly the next smallest distance can be found.

TIME

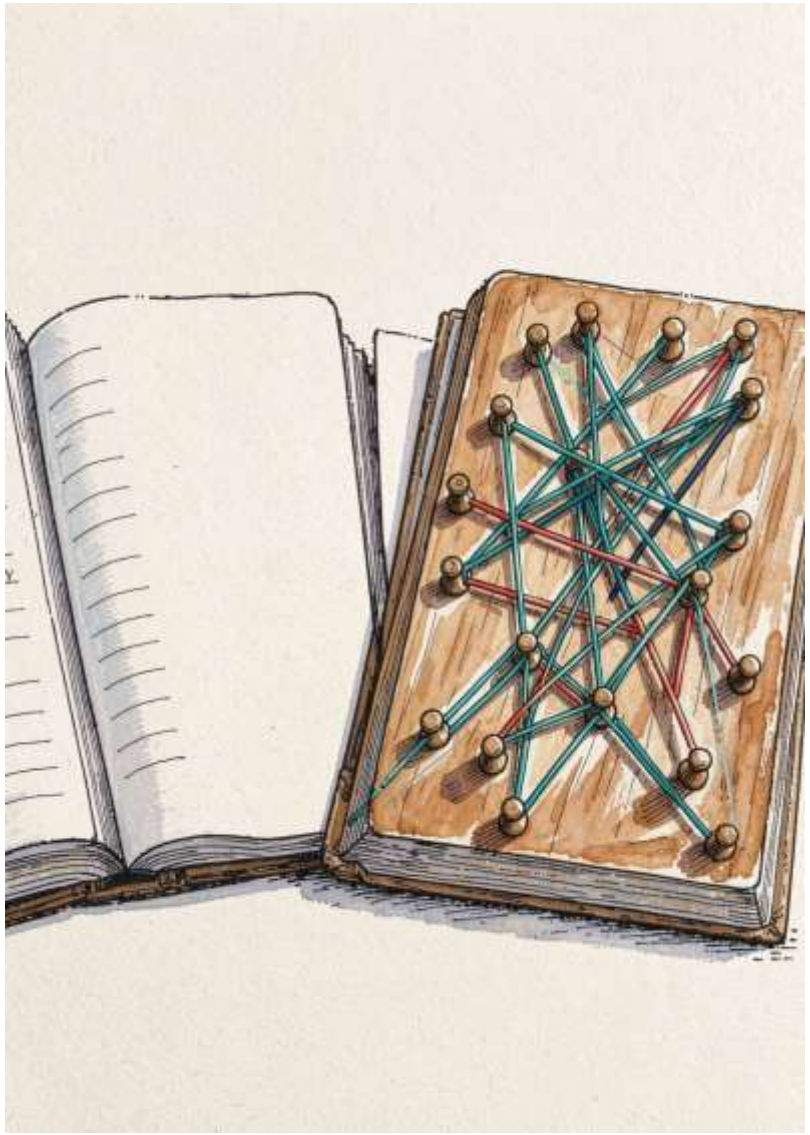
O((V + E) log V)

With an adjacency list and a min-priority queue.

SPACE

O(V + E)

Adjacency-list storage; distance, predecessor, visited, and queue data add O(V).



A good representation makes repeated choices efficient.

Made with GAMMA

Dijkstra's Shortest Path Algorithm

Student: Aanya Mishra
PRN: 25070521279 · Sem/Section: 3/E-2

GUARANTEED SHORTEST ROUTE SETTLED

Thank You

From weighted graphs to optimal decisions through greedy choices and safe relaxation.

STRATEGY	CONDITION	TIME COMPLEXITY
Greedy Selection	Non-Negative Weights	O((V + E) log V)